EBIS Business Techschool

LLM LOCALES · NVIDIA JETSON · OLLAMA

DropLab Analyst

Un modelo de lenguaje local para explicar un problema de negocio, sin que invente nada

Sesión 1 · Teoría y práctica en directo

Malasaña · Goya · Eixample



EBIS Business Techschool

BLOQUE I · EL CASO

El caso DropLab

Un problema de negocio real, en pequeño: qué nos piden y con qué datos contamos.

Pregunta guía
¿Qué ha pasado con las ventas de Malasaña?

EN ESTE BLOQUE

1 **Conoce DropLab**
Una marca de streetwear con tres tiendas, siete modelos y 48 productos.

2 **El encargo**
Un correo urgente: una cifra por comprobar, tres sospechosos y una condición.

3 **Los datos**
Seis archivos: qué representa una fila en cada uno y cómo se conectan.

4 **Trampas y límites**
Contar bien, el JOIN que duplica y lo que no está en los datos.

DropLab: streetwear, tres tiendas y una pregunta por resolver







3 tiendas

Dos en Madrid y una en Barcelona

7 modelos

5 de zapatillas, una sudadera y una camiseta

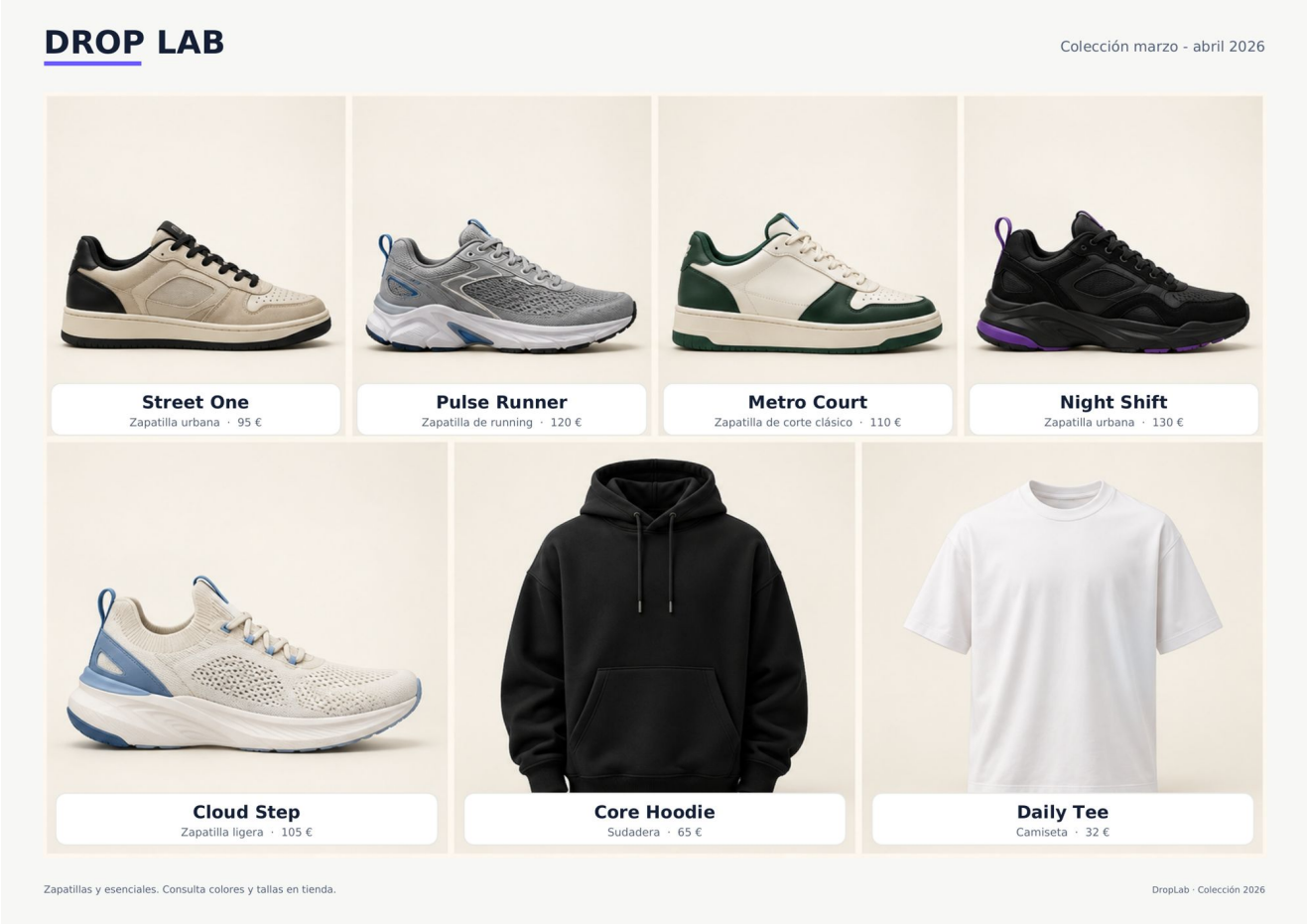
48 productos

Cada combinación de modelo, color y talla

2 periodos

Cuatro semanas de marzo frente a cuatro de abril

Qué vende DropLab: siete modelos, de 32 a 130 €



5 zapatillas

Street One, Pulse Runner, Metro Court, Night Shift y Cloud Step. De 95 a 130 €.

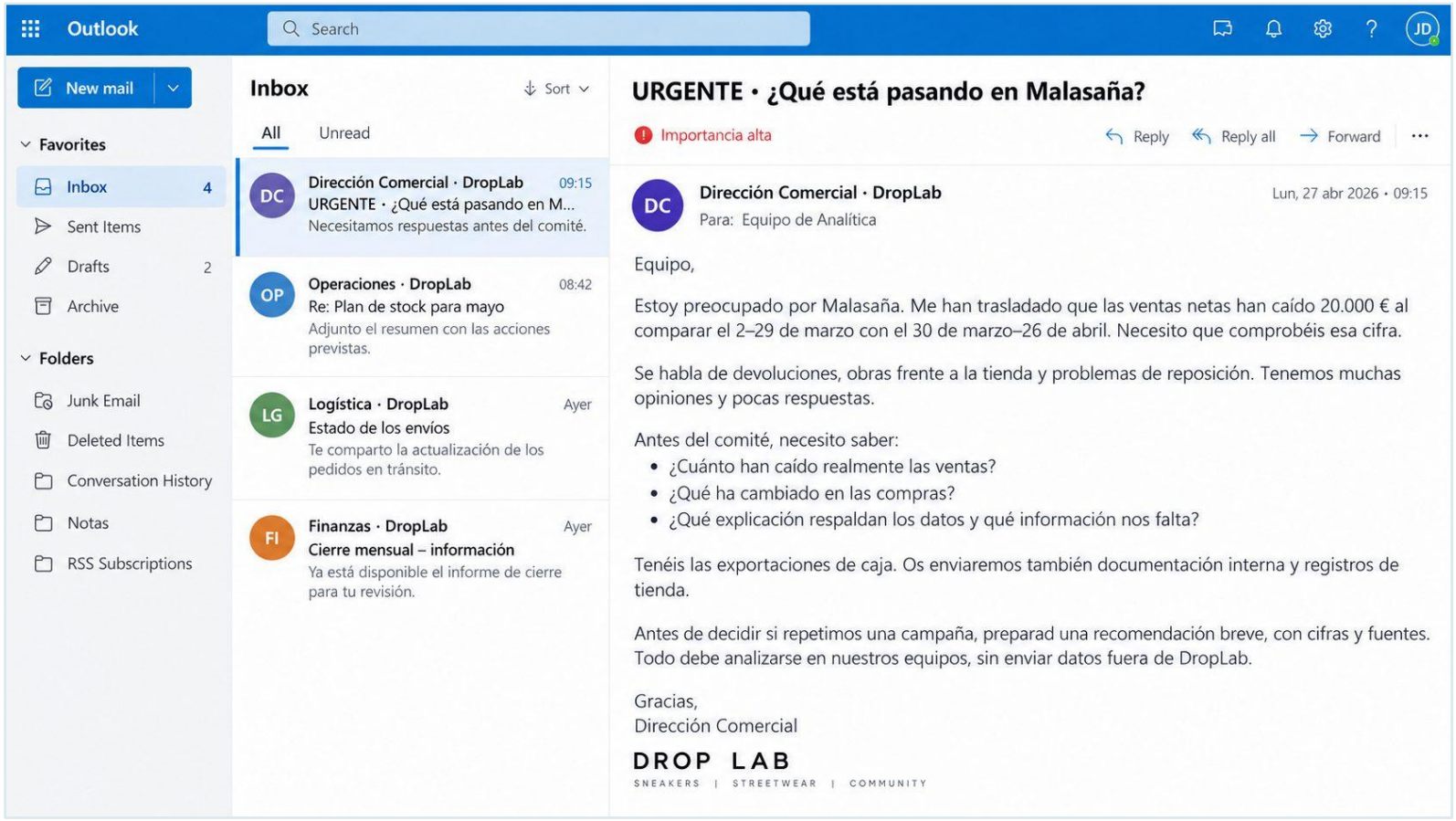
2 prendas

Core Hoodie (sudadera, 65 €) y Daily Tee (camiseta, 32 €).

48 productos

Cada modelo, en 2 colores y varias tallas. Cada combinación es un SKU.

Lunes 27 de abril, 09:15: un correo urgente sobre Malasaña



Una cifra

«Las ventas netas han caído 20.000 €»: se la han trasladado. Hay que comprobarla.

Tres sospechosos

Devoluciones, obras y reposición. Por ahora, opiniones.

Una condición

Analizarlo en nuestros equipos, sin enviar datos fuera.

Lo que nos pide el correo, convertido en un plan

LAS TRES PREGUNTAS DEL CORREO	CÓMO LAS RESPONDEREMOS	
¿Cuánto han caído realmente las ventas?	El código calcula el neto. Un LLM local lo redacta y comprobamos que no invente.	HOY
¿Qué ha cambiado en las compras?	El código cuenta pedidos, líneas y unidades. El LLM pide esos cálculos con herramientas.	HOY
¿Qué explicación respaldan los datos y qué nos falta?	Cruzar las cifras con documentos, tickets e imágenes, y listar lo que falta.	SESIONES 2 Y 3

Pregunta abierta: ¿y si se lo preguntamos directamente a un LLM? **Lo comprobaremos en el bloque II.**

Y DOS CONDICIONES QUE MARCAN EL DISEÑO

«Con cifras y fuentes»
Cada número lo calcula el código y se puede rastrear hasta su archivo.

«Sin enviar datos fuera de DropLab»
El modelo de lenguaje corre en nuestro equipo: un LLM local.

Seis archivos, una historia: los códigos los conectan



Diccionarios: traducen códigos a tiendas, productos y días

Registros: lo que ha pasado (ventas, abonos, stock)

PRÁCTICA 1 · VS Code y 01_inspeccionar_datos.py

locales.csv: tres tiendas y el código que las identifica

locales.csv — 3 filas · 4 columnas

local_id	nombre	ciudad	zona_horaria
MAD-MAL	Malasaña	Madrid	Europe/Madrid
MAD-GOY	Goya	Madrid	Europe/Madrid
BCN-EIX	Eixample	Barcelona	Europe/Madrid

- **Cada fila es una tienda.** DropLab tiene tres: dos en Madrid y una en Barcelona.
- **local_id** es el identificador que encontraremos en el resto de archivos.
- Todas trabajan con la hora de Madrid.

ASÍ LEEMOS UN REGISTRO

en ventas.csv
`local_id = MAD-MAL` → en locales.csv **Malasaña, Madrid**

Es el «traductor» del caso.
Convierte los códigos de los registros en tiendas con nombre.

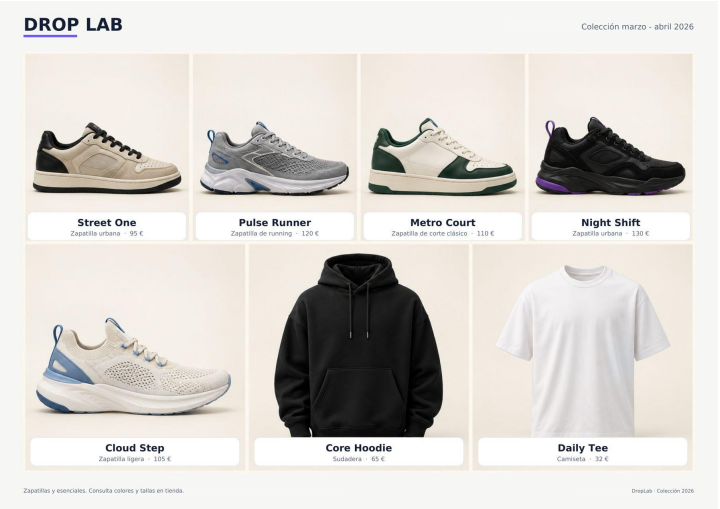
PRÁCTICA 1 · VS Code y 01_inspeccionar_datos.py

referencias.csv: 7 modelos y 48 productos distintos (SKU)

referencias.csv — 48 filas · 7 columnas

sku	modelo	color	categoria	talla	precio_lista
STREET_ONE_NEGRO_40	Street One	negro	zapatilla	40	95.00
STREET_ONE_NEGRO_41	Street One	negro	zapatilla	41	95.00
STREET_ONE_NEGRO_42	Street One	negro	zapatilla	42	95.00
STREET_ONE_NEGRO_43	Street One	negro	zapatilla	43	95.00
STREET_ONE_ARENA_40	Street One	arena	zapatilla	40	95.00
⋮					
CORE_HOODIE_LILA_M	Core Hoodie	lila	sudadera	M	65.00
CORE_HOODIE_LILA_L	Core Hoodie	lila	sudadera	L	65.00

precio_lista no es el coste del producto.
Es el precio de catálogo: no podemos calcular margen ni beneficio.



40 — **Zapatillas = 40 SKU**
5 modelos × 2 colores × 4 tallas (40–43)

8 — **Ropa = 8 SKU**
2 modelos × 2 colores × 2 tallas (M, L)

PRÁCTICA 1 · VS Code y 01_inspeccionar_datos.py

calendario.csv: qué días comparamos exactamente

calendario.csv — 168 filas · 5 columnas

fecha	local_id	abre	cierra	cobertura_completa	
2026-03-28	MAD-MAL	10:00	20:00	true	A
2026-03-29	MAD-MAL	10:00	20:00	true	A
2026-03-30	MAD-MAL	10:00	20:00	true	B
2026-03-31	MAD-MAL	10:00	20:00	true	B

Ojo: el periodo B empieza en marzo, el día 30.
Antes de comparar, fija exactamente qué fechas entran en cada periodo.

Simplificación: todas las tiendas abren de 10:00 a 20:00 cada día; no se modelan festivos ni horarios distintos.

PRÁCTICA 1 · VS Code y 01_inspeccionar_datos.py

LOS DOS PERIODOS



28 días · 4 semanas **28 días · 4 semanas**

- **Una fila = una tienda en un día:** 56 días × 3 tiendas = 168 filas.
- Los dos periodos empiezan en lunes y duran cuatro semanas: comparamos lo mismo con lo mismo.

ventas.csv: cada fila es una línea de un pedido

ventas.csv — 1.441 filas · 10 columnas

linea_id	pedido_id	fecha_venta	sku	cantidad	precio_unitario	descuento_linea	importe_linea
A-MAD-MAL-0014	A-MAD-MAL-ORD-0006	2026-03-08	CORE_HOODIE_LILA_L	2	65.00	14.97	**115.03**
A-MAD-MAL-0015	A-MAD-MAL-ORD-0006	2026-03-08	STREET_ONE_NEGRO_42	2	95.00	21.87	**168.13**
A-MAD-MAL-0016	A-MAD-MAL-ORD-0007	2026-03-09	PULSE_RUNNER_GRIS_41	2	120.00	27.63	**212.37**
A-MAD-MAL-0017	A-MAD-MAL-ORD-0007	2026-03-09	METRO_COURT_BLANCO_40	2	110.00	25.33	**194.67**

Además: local_id y estado_linea. Las dos filas resaltadas son la misma compra.

CÓMO SE CALCULA EL IMPORTE DE UNA LÍNEA

```
cantidad × precio_unitario − descuento_linea
```

2 × 95,00 − 21,87 = 168,13 €

- **1.441 filas:** 720 en A, 720 en B y una venta antigua de febrero.
- El descuento ya está restado: sumar importe_linea da las **ventas antes de reembolsos**.
- fecha_venta tiene día, **no hora**.

PRÁCTICA 1 · VS Code y 01_inspeccionar_datos.py

Una compra, dos líneas, cuatro unidades: contar bien

PEDIDO

A-MAD-MAL-ORD-0006

8 de marzo · Malasaña

2 ×	**Core Hoodie lila L** A-MAD-MAL-0014	115,03 €
2 ×	**Street One negro 42** A-MAD-MAL-0015	168,13 €
Total		**283,16 €**

1

compra

Contamos **pedido_id distintos**

2

líneas

Contamos **filas**

4

unidades

Sumamos la columna **cantidad**

A-MAD-MAL-0014 es una línea; A-MAD-MAL-ORD-0014 es otro pedido.
Se parecen, pero no tienen nada que ver: usa siempre el identificador completo y su columna.

PRÁCTICA 1 · VS Code y 01_inspeccionar_datos.py

reembolsos.csv: cada fila es un abono sobre una venta

reembolsos.csv — 62 filas · 6 columnas

reembolso_id	linea_id	fecha_reconocimiento	unidades	importe_reembolso	motivo
REF-0001	HIST-JOIN-001	2026-03-07	0	10.00	ajuste_comercial
REF-0002	HIST-JOIN-001	2026-03-10	0	20.00	ajuste_comercial
REF-0003	A-MAD-MAL-0002	2026-03-04	2	168.13	talla
REF-0031	A-MAD-MAL-0030	2026-04-01	2	212.37	talla

1 linea_id lleva a la venta

De la venta original sacamos la tienda y el producto: este archivo no los tiene.

2 Cuenta el día del abono

REF-0031 abona el 01/04 una venta del 16/03: la venta va en A y el abono, en B.

3 unidades = 0 también existe

Es un ajuste de dinero sin devolución física (ajuste_comercial).

PRÁCTICA 1 · VS Code y 02_auditoria_integridad.py

Trampa del JOIN: la misma venta, contada dos veces

LO QUE HAY EN LOS ARCHIVOS

ventas.csv

linea_id	fecha_venta	importe_linea
HIST-JOIN-001	2026-02-20	**100.00**

reembolsos.csv

reembolso_id	linea_id	importe_reemb.
REF-0001	HIST-JOIN-001	**10.00**
REF-0002	HIST-JOIN-001	**20.00**

JOIN →

UNIÓN DIRECTA POR linea_id

ventas + reembolsos

linea_id	importe_linea	importe_reemb.
HIST-JOIN-001	100.00	10.00
HIST-JOIN-001	100.00	20.00

suma de importe_linea

200 € ✗

La venta aparece una vez por cada abono.

LO CORRECTO

1. Sumar abonos por línea
10 + 20 = 30 €

2. Después, unir
100 − 30 = 70 € ✓

Y la venta es de febrero: en el periodo A solo cuentan los 30 € de abonos.

Antes de unir dos tablas, agrega la que tiene varias filas por cada fila de la otra.

PRÁCTICA 1 · VS Code y 02_auditoria_integridad.py

disponibilidad.csv: solo vemos el stock de 8 SKU

disponibilidad.csv — 1.352 filas · 6 columnas

local_id	sku	inicio	fin	unidades_disponibles
MAD-MAL	STREET_ONE_NEGRO_40	2026-04-06T10:00+02:00	2026-04-06T20:00+02:00	4
MAD-MAL	STREET_ONE_NEGRO_41	2026-04-06T10:00+02:00	2026-04-06T16:00+02:00	0
MAD-MAL	STREET_ONE_NEGRO_41	2026-04-06T16:00+02:00	2026-04-06T20:00+02:00	3
MAD-MAL	STREET_ONE_NEGRO_42	2026-04-06T10:00+02:00	2026-04-06T16:00+02:00	0

Además: estado_registro (siempre «observado»). Hora local con su desfase (+02:00); en la captura se omiten los segundos.

Una fila = un tramo de horas

Un SKU, en una tienda, entre inicio y fin, con sus unidades. El inicio cuenta; el fin, no.

Solo 8 SKU, todos Street One

2 colores × 4 tallas, en las 3 tiendas. Los otros 40 productos no se observan.

Sin fila no es stock cero

Significa que no lo medimos. Y no es un libro de almacén: no cuadra entradas ni ventas.

PRÁCTICA 1 · VS Code y 01_inspeccionar_datos.py

Lo que no está en los datos tampoco se puede concluir

✗ Clientes

No hay cliente_id: un pedido no es una persona.

✗ Visitas

Sin tráfico ni conversión: no sabemos cuánta gente entró.

✗ Hora de venta

Solo el día: no podemos cruzar ventas con las horas sin stock.

✗ Costes y margen

Solo precios de venta: no se puede calcular el beneficio.

✗ Uso de campañas

No sabemos qué pedidos se acogieron a una promoción.

✗ Demanda no atendida

No sabemos qué se quiso comprar y no había.

Un dato ausente no es un cero: **si hace falta, se pide; no se inventa**.

PRÁCTICA 1 · VS Code y 01_inspeccionar_datos.py

EBIS Business Techschool

PRÁCTICA 1 · BLOQUE I

Los datos, en VS Code

Ver con nuestros ojos lo que acabamos de explicar: filas, códigos y trampas.

▸ Ahora, en la terminal

AL TERMINAR

Bloque II: qué es un LLM y cómo funciona en local.

PASOS

1 Abrir los seis CSV

```
code data/datos
```

→ Rainbow CSV colorea cada columna. ¿Qué es una fila en cada archivo?

2 Una compra completa (RBQL sobre ventas.csv)

```
SELECT * WHERE a2 == 'A-MAD-MAL-ORD-0006'
```

→ Dos líneas, cuatro unidades, un único pedido.

3 La trampa del JOIN (RBQL sobre ventas.csv)

```
SELECT a1, COUNT(*), SUM(parseFloat(a9)) INNER JOIN reembolsos.csv ON a1
== b2 WHERE a1 == 'HIST-JOIN-001' GROUP BY a1
```

→ Dos filas y 200 €: la venta de 100 € aparece una vez por cada abono.

4 Lo confirma el código

```
python practica/01_inspeccionar_datos.py && python
practica/02_auditoria_integridad.py
```

→ Filas y columnas de cada archivo, cero huérfanos y la misma trampa: 200 € en vez de 100.

EBIS Business Techschool

LLM LOCALES · NVIDIA JETSON · OLLAMA

DropLab Analyst

Un modelo de lenguaje local para explicar un problema de negocio, sin que invente nada

Sesión 1 · Teoría y práctica en directo

Malasaña · Goya · Eixample



EBIS Business Techschool

BLOQUE II · LLM LOCALES

LLM locales

Un repaso rápido de lo que ya sabéis y cómo se ejecuta un modelo en nuestro propio equipo.

Pregunta guía
¿Puede ayudarnos un modelo sin sacar los datos de la empresa?

EN ESTE BLOQUE

1 **Repaso de LLM**
Lo justo para hoy: tokens, parámetros y contexto.

2 **Nube o local**
Hugging Face, Ollama y las familias de modelos abiertos.

3 **Ollama, en directo**
Descargar, preguntarle por DropLab y ponerlo a prueba.

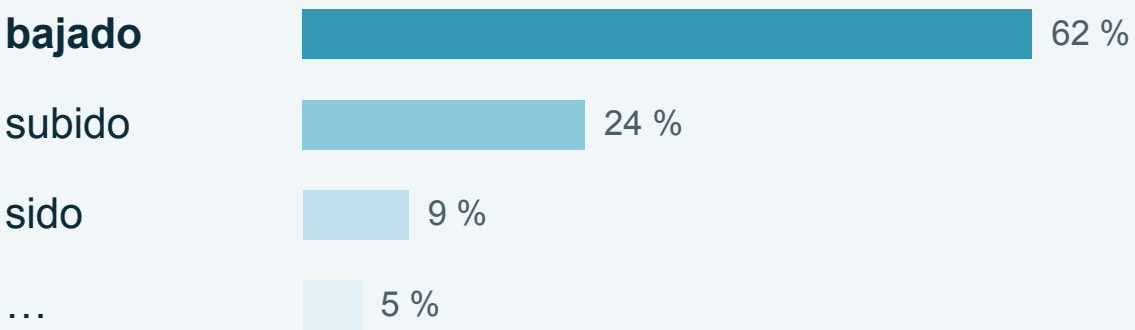
4 **Elegir modelo**
Nombre, cuantización, variante y ventana de contexto.

Repaso: un LLM es un autocompletado gigante

ESCRIBES

Las ventas de abril han **?**

EL MODELO PUNTÚA CADA PALABRA POSIBLE



…y lo repite, palabra a palabra, hasta terminar la respuesta.

Token
Un trozo de texto. Una palabra ocupa uno o varios tokens.

Parámetros
Lo que aprendió al entrenar. Nuestro modelo tiene unos 4.000 millones.

Contexto
Todo lo que tiene delante a la vez: instrucciones, datos y su respuesta.

Nube o local: la misma tecnología en distinto sitio

EN LA NUBE

ChatGPT · Claude · Gemini

Modelos cerrados: solo se usan por Internet

- ✓ Los modelos más potentes
- ✓ Nada que instalar
- ✗ Tus datos viajan a otra empresa
- ✗ Pago por uso y siempre con Internet

EN LOCAL

Llama · Qwen · Gemma…

Modelos abiertos: se descargan y corren en tu equipo

- ✓ Los datos no salen del equipo
- ✓ Sin coste por uso; funciona sin red
- ✗ Modelos más pequeños
- ✗ Necesitas memoria y, mejor, una GPU

En DropLab elegimos **local**: son datos de negocio que no deben salir de la empresa.

Hugging Face es el almacén; Ollama, el reproductor

1

Hugging Face

El «GitHub» de los modelos: ahí se publican más de un millón.

huggingface.co

listos para usar (GGUF)

2

Ollama

Descarga el modelo una vez, lo carga en memoria y responde.

ollama run qwen3:4b-instruct

pregunta y respuesta

3

Tu aplicación

Le pregunta a Ollama por HTTP, como a cualquier API.

localhost:11434

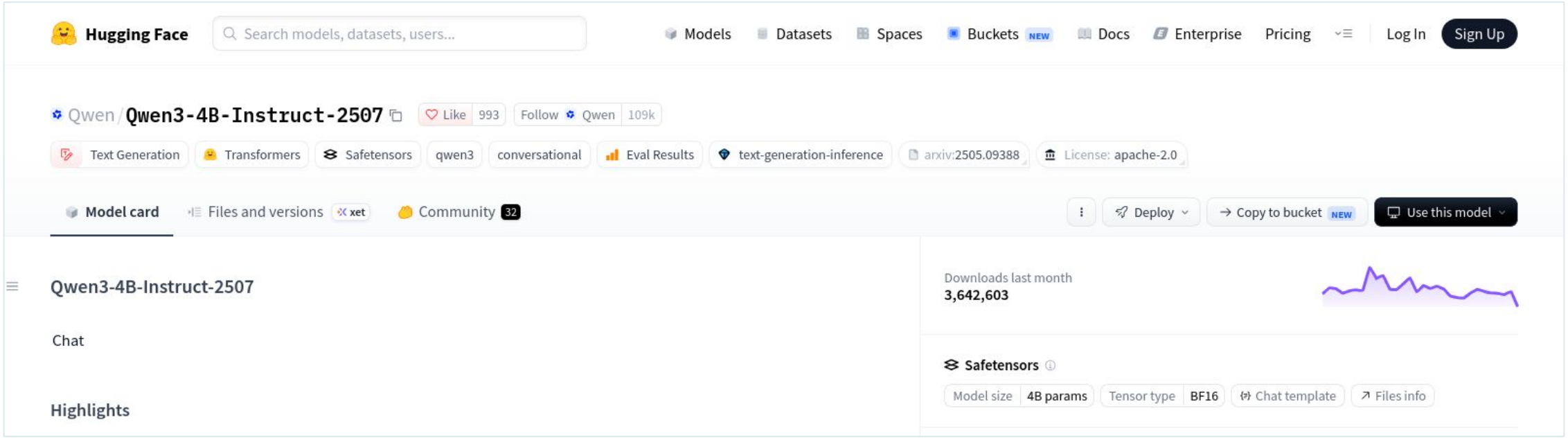
OTRAS HERRAMIENTAS HABITUALES

LM Studio
Como Ollama, pero con ventanas y botones.

llama.cpp
El motor abierto sobre el que nació Ollama.

vLLM
Para servir modelos a muchos usuarios en servidores.

Así es Hugging Face: la ficha de nuestro modelo



QUÉ MIRAR EN LA FICHA

Qwen / Qwen3-4B-Instruct-2507
Quién lo publica (Alibaba) y su nombre completo.

3,6 millones de descargas
Solo el último mes: un modelo muy usado.

4B params · BF16
4.000 millones de parámetros en 16 bits. Ollama lo baja a 4.

License: apache-2.0
Se puede usar en una empresa, también para vender.

Text Generation · conversational
Las etiquetas: qué tarea hace el modelo.

Los modelos abiertos más conocidos

Se descargan gratis y se usan en local, cada uno con su licencia. El panorama cambia cada pocos meses.

Llama

Meta · EE. UU.

1B 3B 8B 70B

Pionero de los modelos abiertos. Lo usamos como modelo pequeño.

Qwen

EL NUESTRO

Alibaba · China

0,6B 4B 8B 32B

Multilingüe y muy bueno en español. Es nuestro modelo.

Gemma

Google · EE. UU.

1B 4B 12B 27B

Ligero; los medianos también ven imágenes.

Mistral

Mistral AI · Francia

7B 24B

El referente europeo.

DeepSeek

DeepSeek · China

1,5B 8B 70B

Famoso por razonar paso a paso (R1).

gpt-oss

OpenAI · EE. UU.

20B 120B

El primer modelo abierto de OpenAI desde GPT-2.

EBIS Business Techschool

PRÁCTICA 2 · BLOQUE II

Ollama, en directo

Descargar un modelo, preguntarle por DropLab y ponerlo a prueba.

▶ Ahora, en la terminal

AL TERMINAR
Las slides siguientes explican lo que acabamos de ver.

PASOS

1 Qué hay instalado y cómo se descarga

```
ollama list && ollama pull qwen3:4b-instruct
```

→ Se instala con un solo comando. El pull acaba al instante: ya lo tenemos.

2 ¿Sabe algo de DropLab?

```
ollama run qwen3:4b-instruct "¿Cuánto cayeron las ventas de la tienda de Malasaña de DropLab entre marzo y abril de 2026?"
```

→ No conoce la empresa ni nuestros datos (y cree que estamos en 2024).

3 ¿Sabe calcular?

```
ollama run qwen3:4b-instruct "Responde solo: 17*23"
```

→ Contesta 401, con total seguridad. La respuesta correcta es 391.

4 Qué modelo es y cuánta memoria usa

```
ollama show qwen3:4b-instruct && ollama ps
```

→ 4.0B parámetros, Q4_K_M y ventana de 262.144 tokens. En ps: SIZE y CONTEXT.

Un LLM predice palabras. No calcula.

LE PREGUNTAMOS CINCO VECES

17 × 23

*«Responde solo: 17*23»*

EL MODELO RESPONDIÓ

401 401 401 401 401

La respuesta correcta es

391

Suena seguro, es falso y no avisa.

Por eso las cifras las calcula el código, nunca el modelo.

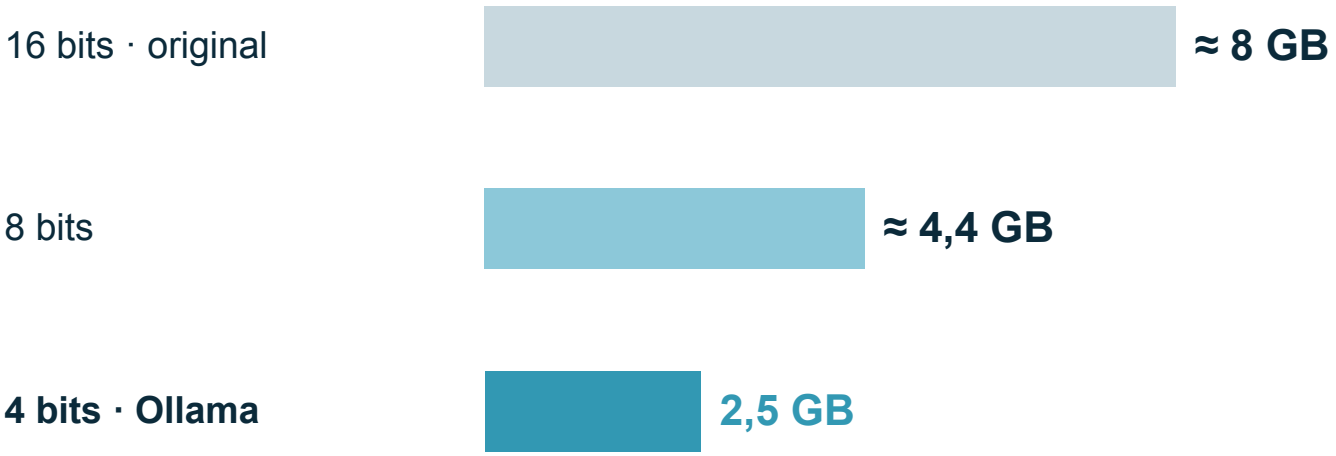
Cómo leer el nombre de un modelo

qwen3	:4b	-instruct	-q4_K_M
FAMILIA	**TAMAÑO**	**VARIANTE**	**COMPRESIÓN**
Qwen, versión 3	4.000 millones de parámetros	Responde directo. Otras: thinking, vision…	Pesos guardados en ≈ 4 bits

Regla rápida: **más parámetros = más capaz**, pero también más lento y con más memoria. Se elige el más pequeño que haga bien la tarea.

Cuantizar: comprimir el modelo para que quepa en tu equipo

EL MISMO MODELO (qwen3:4b), SEGÚN CÓMO SE GUARDE



Como bajar la resolución de una foto: **pesa la tercera parte** y se sigue reconociendo.

¿QUÉ CABE EN MI EQUIPO?

Portátil de 8 GB
modelos de 1B a 4B

Portátil 16 GB o GPU de 8 GB
hasta unos 8B

Jetson Orin 64 GB
8B con holgura, hasta unos 30B

Orientativo, en 4 bits. Hay que sumar el contexto: lo vemos enseguida.

Todo ocurre en el Jetson: los datos no salen del equipo

NVIDIA JETSON ORIN · 64 GB

Nuestro código

`Python · pandas`

→

Ollama

`localhost:11434`

→

Modelo

`qwen3:4b-instruct`

Pregunta → Ollama → modelo → respuesta. **Ningún paso usa Internet.**

Internet

Solo una vez, para descargar el modelo.

En directo

Desconectamos la red y el modelo sigue respondiendo.

PRÁCTICA 2 · ollama list · run · ps

Elegir bien la variante: de más de 3 minutos a 2 segundos

PIENSA ANTES DE RESPONDER

`qwen3:4b`

> 180 s

Genera más de 1.200 tokens de razonamiento oculto antes de contestar.

RESPONDE DIRECTAMENTE

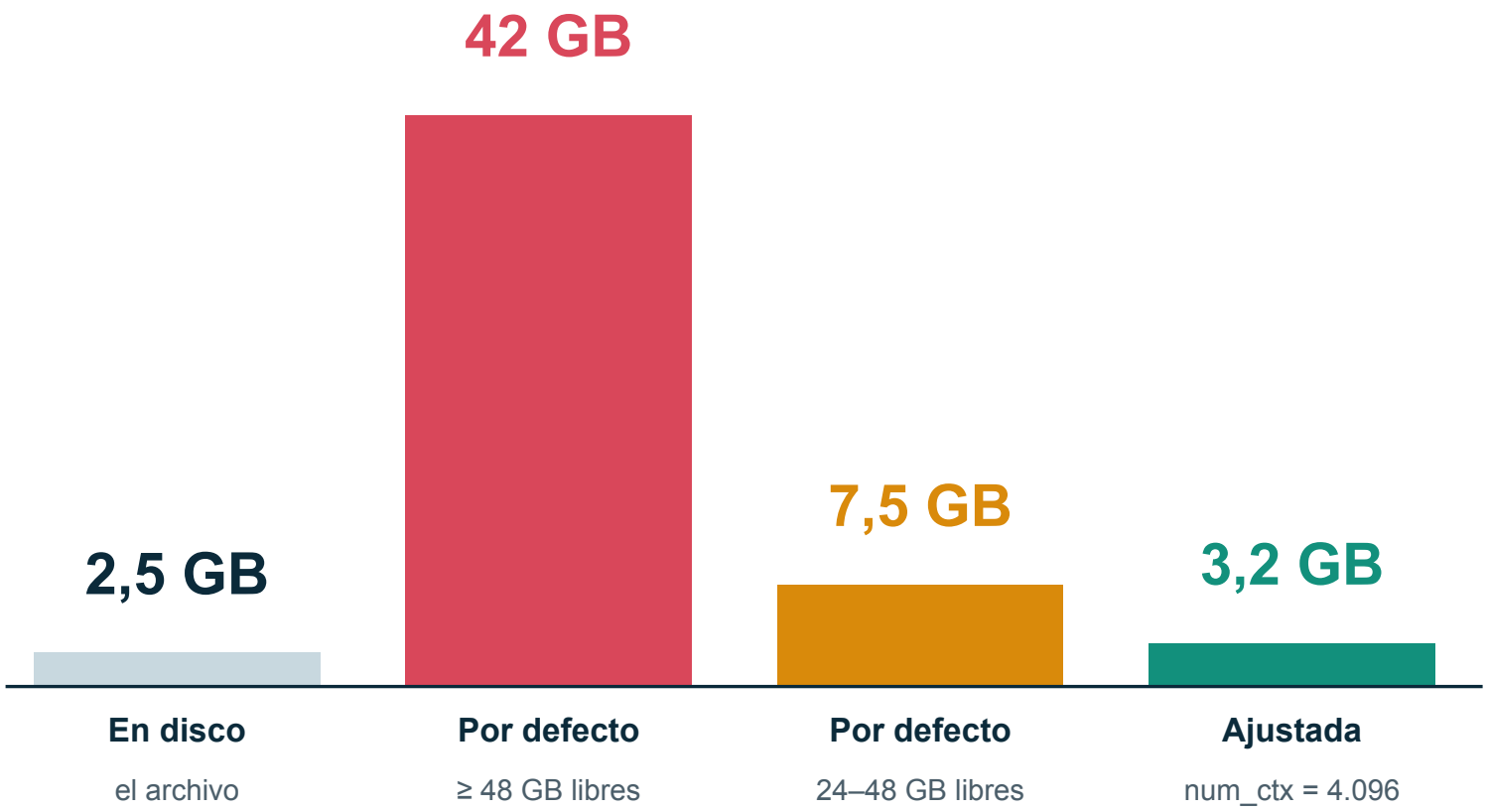
`qwen3:4b-instruct`

2 s

Mismo tamaño (2,5 GB). Ideal para redactar sobre cifras ya calculadas.

Antes de usar un modelo, comprobamos qué hace: en **/api/show**, el campo **thinking**.

Lo que más memoria gasta no es el modelo: es el contexto



¿Qué es el contexto?

La «memoria de trabajo» de la conversación. Ollama la reserva entera al cargar el modelo, se use o no.

Si no la fijas, Ollama la elige según la memoria libre: 262.144 tokens con 48 GB o más; 32.768 entre 24 y 48. El Jetson está en el límite.

Nuestra petición usa ≈ 1.600 tokens. Fijamos num_ctx = 4.096.

PRÁCTICA 2 · ollama ps (columna CONTEXT)

EBIS Business Techschool

LLM LOCALES · NVIDIA JETSON · OLLAMA

DropLab Analyst

Un modelo de lenguaje local para explicar un problema de negocio, sin que invente nada

Sesión 1 · Teoría y práctica en directo

Malasaña · Goya · Eixample



EBIS Business Techschool

BLOQUE III · LAS CIFRAS

Las cifras

El modelo no calcula, así que calculamos nosotros: las dos primeras preguntas del correo.

Pregunta guía
¿Cuánto han caído realmente las ventas y qué ha cambiado?

EN ESTE BLOQUE

1. **Calcular con código**
   Ventas, reembolsos y neto: el script 03 lo hace en un segundo.
2. **Dónde cae la venta**
   El total de la empresa y el peso de cada tienda.
3. **La cifra del correo**
   Los 20.000 € frente a lo que dicen los datos.
4. **Los sospechosos**
   Devoluciones, compras y stock: qué respaldan los datos y qué no.

EBIS Business Techschool

PRÁCTICA 3 · BLOQUE III

Las cifras, con código

Responder a las dos primeras preguntas del correo en un segundo y sin errores.

▸ Ahora, en la terminal

AL TERMINAR
Las slides siguientes interpretan cada parte de la salida.

PASOS

1 Calcular todo

```
python practica/03_metricas_basicas.py
```

→ Total: 96.000 → 81.000 €. Malasaña: 40.000 → 26.000 € (−14.000 €).

2 Leer la cesta

→ Malasaña: 120 pedidos en A y en B, pero 3,89 → 2,53 unidades por pedido.

3 Leer la disponibilidad

→ 8 intervalos sin stock: 48 horas-SKU, pero solo 12 horas de reloj.

4 ¿Y con RBQL? (ventas.csv)

```
SELECT a4, SUM(parseFloat(a9)) WHERE a3 >= '2026-03-02' && a3 <=
'2026-03-29' GROUP BY a4
```

→ 42000.00000000003: por eso 03 suma en céntimos.

Primero, el total: las ventas netas de DropLab caen un 15,6 %







96.000 €

Periodo A · 2–29 marzo 2026

→

81.000 €

Periodo B · 30 marzo–26 abril 2026

−15,6 %

−15.000 € en ventas netas

PRÁCTICA 3 · 03_metricas_basicas.py

El 93 % de la caída está en una sola tienda: Malasaña

Malasaña

40.000 → 26.000 €



−14.000 € (−35 %)

Goya

32.000 → 30.000 €



−2.000 € (−6 %)

Eixample

24.000 → 25.000 €



+1.000 € (+4 %)

93 %

de la caída total viene de Malasaña

Esto nos dice **dónde** cae la venta. Todavía no sabemos **por qué**.

El correo decía −20.000 €. Los datos dicen −14.000 €

LO QUE NOS DIJERON

−20.000 €

Correo de Dirección Comercial: «me han trasladado…»

→

LO QUE DICEN LOS DATOS

−14.000 €

Malasaña, ventas netas: 40.000 → 26.000 €

¿CUADRAN LOS 20.000 € CON ALGUNA MÉTRICA?

Malasaña · bruto	**−14.000 €**	≠ 20.000
Malasaña · neto	**−14.000 €**	≠ 20.000
DropLab · neto total	**−15.000 €**	≠ 20.000

Una cifra que nos llega es una **hipótesis** hasta que la comprobamos con los datos.

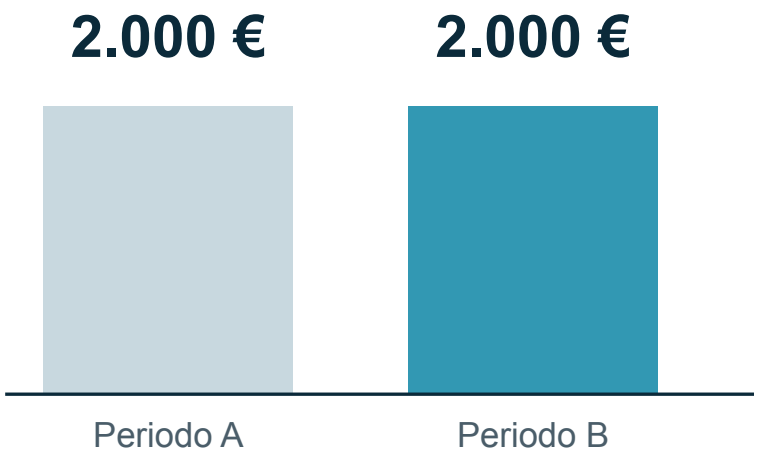
PRÁCTICA 3 · 03_metricas_basicas.py

¿Son las devoluciones? Los datos dicen que no

SOSPECHOSO 1 · DEL CORREO

«Se habla de devoluciones…»

ABONOS RECONOCIDOS EN MALASAÑA



Mismo importe en A y en B: explican 0 € de los 14.000 € de caída.

LA TRAMPA DEL PORCENTAJE

4,76 % → 7,14 %

abonos sobre ventas antes de reembolsos

El porcentaje sube, pero los euros no.

Sube porque baja el denominador: 42.000 → 28.000 €. Di siempre sobre qué se calcula un %.

PRÁCTICA 3 · 03_metricas_basicas.py

Mismos pedidos, menos unidades en cada compra

PEDIDOS

A 120

B 120

igual

UNIDADES

A 467

B 304

−35 %

UNIDADES POR PEDIDO

A 3,89

B 2,53

−35 %

TICKET MEDIO

A 350,00 €

B 233,33 €

−33 %

Mismo número de compras, menos artículos en cada una.
El precio de lista medio no cambia: no es un efecto de precios.

Un pedido no es una persona.
Sin datos de visitas, no sabemos si entró menos gente ni quién compró.

PRÁCTICA 3 · 03_metricas_basicas.py

Una pista: faltó stock en Malasaña. No prueba la causa.



El resto del periodo B, con unidades disponibles. Goya y Eixample: ningún hueco.

8 intervalos sin stock

48 h horas-SKU: 4 SKU × 6 h × 2 días

12 h de reloj: el 4 % del horario de B

Coincidir en el tiempo no es causar.
12 horas en 4 SKU no explican por sí solas un mes de caída en todo el catálogo.

EBIS Business Techschool

LLM LOCALES · NVIDIA JETSON · OLLAMA

DropLab Analyst

Un modelo de lenguaje local para explicar un problema de negocio, sin que invente nada

Sesión 1 · Teoría y práctica en directo

Malasaña · Goya · Eixample



EBIS Business Techschool

BLOQUE IV · EL MODELO EN ACCIÓN

El modelo en acción

Conectamos el modelo con los datos y lo ponemos a prueba.

Regla de oro
El código calcula, el modelo redacta y tú verificas.

EN ESTE BLOQUE

1 **El modelo redacta**
Cuatro pruebas: qué necesita para no inventar.

2 **Dónde se equivoca**
Errores que suenan bien y no avisan.

3 **Qué recibe exactamente**
El modelo nunca ve los CSV: solo cifras ya calculadas.

4 **Herramientas**
El modelo pide los cálculos: función fija frente a SQL libre.

La regla de oro de todo el curso



El código calcula.

pandas y DuckDB: exacto y repetible.



El modelo redacta.

Convierte cifras en un texto claro.



Tú verificas.

Cada cifra del texto, contra los hechos.

EBIS Business Techschool

PRÁCTICA 4 · BLOQUE IV

El modelo redacta

Misma pregunta, cuatro pruebas.
Cambiamos una sola cosa cada vez:
¿qué necesita para no inventar?

▶ Ahora, en la terminal

AL TERMINAR
Resumen en slides y, al final, qué recibe exactamente el modelo (06).

PASOS

1 **Prueba 1 · Solo la pregunta**

```
python practica/04_conectar_ollama.py --modo sin-datos --sin-reglas
```

→ Le damos un papel y nada más. ¿Qué cifra da? ¿De dónde la saca?

2 **Prueba 2 · + el correo**

```
python practica/04_conectar_ollama.py --modo correo --sin-reglas
```

→ ¿Comprueba los 20.000 € o los repite? ¿Qué fuente cita?

3 **Prueba 3 · + los hechos calculados**

```
python practica/04_conectar_ollama.py --modo hechos --sin-reglas
```

→ Las cifras, bien. ¿Y las causas? ¿Y el 7,14 %?

4 **Prueba 4 · + las reglas**

```
python practica/04_conectar_ollama.py
```

→ Cifras correctas. ¿Cuántos SKU sin stock dice? ¿Son 8?

Misma pregunta, distinto contexto: sin hechos inventa

	1 · SOLO LA PREGUNTA	2 · + EL CORREO	3 · + LOS HECHOS CALCULADOS
SIN REGLAS	*«Las ventas netas de Malasaña han disminuido un 15 %… se debe principalmente a una reducción en el número de compras por parte de clientes nuevos.»* ✗ **Inventa la cifra y la causa**	*«Las ventas netas de Malasaña han caído en 20.000 euros… según los registros de exportación del sistema DropLab.»* ✗ **Repite el correo como si fuera un dato**	*«…han caído realmente 14.000 euros, un 35 %… Las devoluciones representan un 7,14 %… lo que indica una mayor incidencia de calidad…»* ≈ **Cifras bien, causas inventadas**
CON REGLAS	*«No se puede determinar el descenso de ventas ni el cambio en las compras con los datos disponibles.»* ✓ **Admite que no lo sabe**	*«No se puede confirmar si las ventas han caído realmente en 20.000 euros… No se ofrecen fuentes para verificar el correo.»* ✓ **No da por buena la cifra**	*«…han caído realmente en 14.000 euros (35 %)… las unidades de 467 a 304, con un ticket medio que ha bajado de 350 a 233,33 euros.»* ✓ **Cifras correctas**

PRÁCTICA 4 · 04_conectar_ollama.py --modo

Si le falta un dato, lo dice. Si lo tiene, no exagera.

«¿Por qué cayeron las ventas en Malasaña? ¿Fue por falta de stock disponible?»

SIN EL DATO DE STOCK

«No se puede confirmar si fue por falta de stock disponible, ya que no se proporcionan datos sobre disponibilidad de stock o demanda no atendida.»

✓ **Admite el hueco en vez de inventar**

CON EL DATO DE STOCK

«…principalmente en el modelo Street One y tallas 41 y 42, durante dos fechas específicas. No se puede afirmar que la falta de stock sea la causa directa, ya que no hay datos sobre demanda no atendida o tráfico.»

✓ **Habla de coincidencia, no de causa**

PRÁCTICA 4 · 04_conectar_ollama.py -q … --sin-stock

Pero también se equivoca, y no avisa

EL MODELO ESCRIBIÓ

«La disponibilidad de stock cero afectó a…»

8 SKU

El 8 existe en los hechos: son los intervalos y los SKU observados.

→

EL DATO REAL

Negro y arena, tallas 41 y 42

4 SKU

El revisor automático no lo detecta: el número existe, el significado no.

Comprobar que una cifra existe no basta: hay que comprobar **a qué se refiere**.

El modelo nunca ve los CSV, solo cifras ya calculadas

1

Datos

6 ficheros CSV

`data/datos/`

2

Cálculo

pandas calcula las cifras

`03_metricas_basicas`

3

Hechos

un JSON pequeño con los resultados

`construir_evidencia()`

4

Redacción

el modelo escribe el texto

`04_conectar_ollama`

5

Informe

hechos y borrador, por separado

`06_flujo_visual`

HECHOS
Siempre iguales, calculados con código.

BORRADOR
Texto del modelo: se revisa antes de usarlo.

PRÁCTICA 4 · 06_flujo_visual.py

EBIS Business Techschool

PRÁCTICA 5 · BLOQUE IV

El modelo pide herramientas

El modelo decide qué cálculo hace falta; nuestro código lo ejecuta.

▸ **Ahora, en la terminal**

AL TERMINAR
Las slides siguientes comparan función fija y SQL libre.

PASOS

1 Función fija

```
python practica/07_herramienta_funcion.py
```

→ Pide dos llamadas, una por periodo, con las fechas correctas.

2 Esquema y barreras

```
python practica/08_herramienta_sql.py --solo-esquema
```

→ El esquema se escribe solo; DELETE y leer archivos quedan bloqueados.

3 SQL libre con reglas

```
python practica/08_herramienta_sql.py
```

→ Lee el SQL antes que el resultado: ¿qué fechas usa para el periodo A?

4 SQL libre sin reglas

```
python practica/08_herramienta_sql.py --sin-reglas
```

→ Se ejecuta sin error… y calcula otra cosa.

Función fija: el modelo pide, nuestro código calcula

QWEN3:4B-INSTRUCT · «¿CUÁNTO HAN CAÍDO LAS VENTAS NETAS DE MALASAÑA…?»

EL MODELO PIDE

```
resumen_comercial("MAD-MAL", "2026-03-02", "2026-03-29")
```

NUESTRO CÓDIGO CALCULA `ventas_netas_eur: 40000.0 · pedidos: 120`

EL MODELO PIDE

```
resumen_comercial("MAD-MAL", "2026-03-30", "2026-04-26")
```

NUESTRO CÓDIGO CALCULA `ventas_netas_eur: 26000.0 · pedidos: 120`

«Las ventas netas de Malasaña han caído de 40.000 € en el periodo A a 26.000 € en el periodo B. En ambos periodos hubo 120 pedidos.»

Fiable
El cálculo está escrito y probado antes: el modelo solo elige cuándo usarlo.

Limitada
Solo responde lo que la función cubre.

PRÁCTICA 5 · 07_herramienta_funcion.py

SQL libre: responde a todo… y puede fallar sin dar error

4B · CON REGLAS

Toma «marzo» como periodo A: mete el 30 y el 31, que son de B

42.332,73 €

✗ Real: 40.000 €. Y olvida el B

4B · SIN REGLAS

JOIN ventas–calendario por tienda: cada venta × 28 días

1.962.800 €

✗ Sin error de DuckDB

8B · CON REGLAS

Fechas bien, pero solo cuenta abonos de ventas del mismo periodo

40.030 / 26.212 €

✗ El grande casi acierta… y falla

BARRERAS EN EL CÓDIGO, NO EN EL PROMPT

```
DELETE FROM ventas
```
✗ bloqueado: solo SELECT

```
read_csv('/etc/passwd')
```
✗ bloqueado: sin acceso a archivos

```
SELECT 1; SELECT 2
```
✗ bloqueado: una sola sentencia

PRÁCTICA 5 · 08_herramienta_sql.py

EBIS Business Techschool

LLM LOCALES · NVIDIA JETSON · OLLAMA

DropLab Analyst

Un modelo de lenguaje local para explicar un problema de negocio, sin que invente nada

Sesión 1 · Teoría y práctica en directo

Malasaña · Goya · Eixample



EBIS Business Techschool

BLOQUE V · COMPARAR Y CERRAR

Comparar y cerrar

Qué modelo y qué herramienta elegir, qué entregamos y qué nos llevamos.

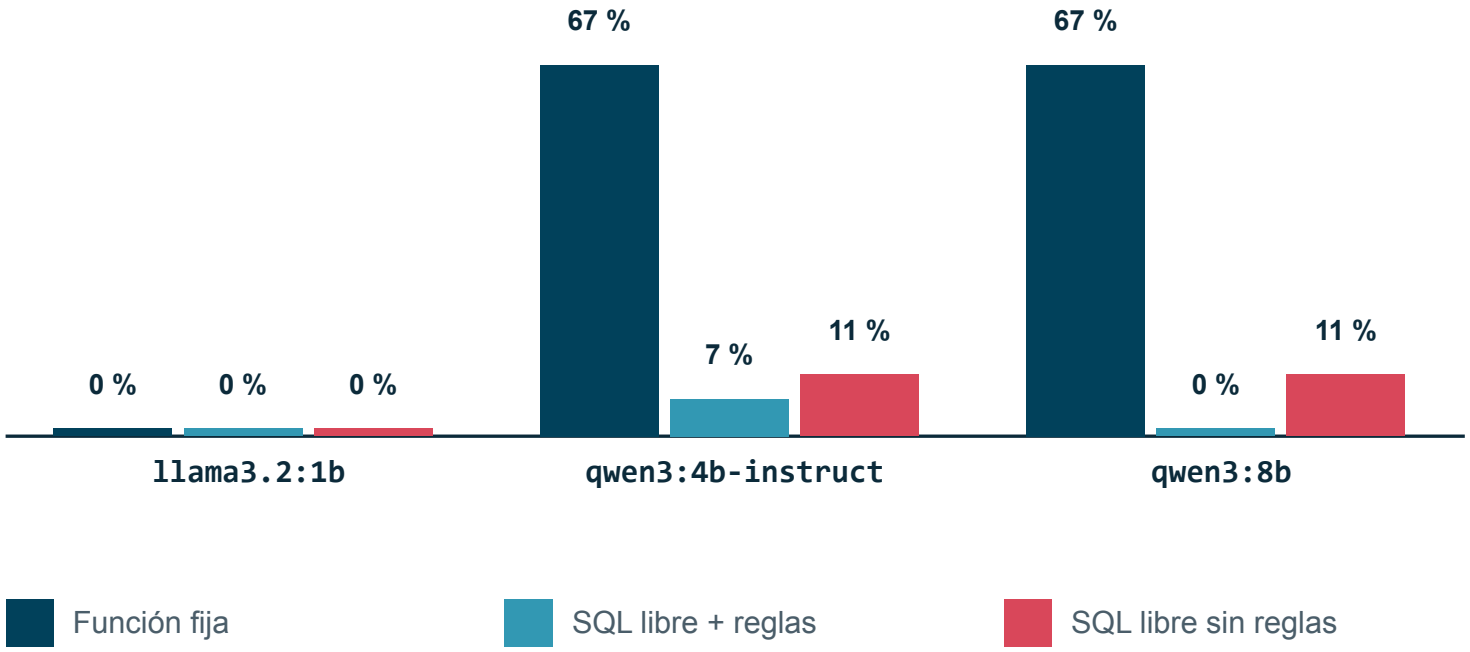
Para casa
Tres ejercicios: montar el laboratorio, revisar y construir.

EN ESTE BLOQUE

1. **Comparar con nota**
   Tres modelos, tres formas de acceder a los datos y su coste.
2. **Cuatro frases para dirección**
   Hechos, interpretación, hipótesis y límites.
3. **Deberes**
   Montar, revisar y construir: unas dos horas.
4. **Para llevar**
   Tres ideas y la pregunta que abre la sesión 2.

No acierta el modelo más grande, sino la mejor herramienta

CAMPOS CORRECTOS · Q01 NETO · Q02 CESTA · Q03 HIST-JOIN-001 · 2 REPETICIONES



COSTE

`llama3.2:1b`
1,3 GB en disco · 1,7 GB cargado · 9 s por pregunta

`qwen3:4b-instruct`
2,5 GB en disco · 3,9 GB cargado · 25 s por pregunta

`qwen3:8b`
5,2 GB en disco · 6,3 GB cargado · 39 s por pregunta

La función acierta lo que cubre y no inventa lo que no cubre (Q03).

EBIS Business Techschool

PRÁCTICA 6 · BLOQUE V

El coste, en directo

Opcional, si hay tiempo: cuánta memoria y GPU gasta cada modelo mientras responde.

▶ Ahora, en la terminal

AL TERMINAR
Cerramos: cuatro frases para dirección.

PASOS

1 Abrir el monitor, en otra terminal

```
python practica/05_monitor_recursos.py
```

→ RAM, GPU y CPU del Jetson en vivo. Antes: ollama stop qwen3:4b-instruct.

2 El modelo mediano

```
python practica/04_conectar_ollama.py
```

→ Al cargar, la RAM sube ~3 GB; al escribir, la GPU se dispara. ~8 tokens/s.

3 El modelo grande

```
python practica/04_conectar_ollama.py --modelo qwen3:8b
```

→ Unos 6 GB más y más lento (~5 tokens/s). ¿Escribe mejor?

Para cerrar: cuatro frases para dirección, con cifras y límites

1

Hechos

Malasaña pasa de 40.000 a 26.000 € netos: −14.000 €, no 20.000.

2

Interpretación

Mantiene 120 pedidos, pero baja de 3,89 a 2,53 unidades por pedido.

3

Hipótesis

Los abonos no cambian; 12 horas sin stock son una pista menor. Falta saber qué cambió el 30 de marzo.

4

Límites

No hay visitas, campañas aplicadas ni demanda no atendida.

La mejor respuesta no es la más rotunda, sino la más **exacta, trazable y honesta**.

Deberes: tres ejercicios, de montar a construir (~2 horas)

EJERCICIO 1

Monta tu laboratorio

~45 min + descarga · Win, Mac o Linux

- Revisa tu equipo (RAM, disco, GPU) y elige modelo.
- Instala Python, el entorno virtual y Ollama.
- Repite lo de clase: cifras, preguntas y script 04.

Lo de hoy, en tu ordenador.

EJERCICIO 2

¿Lo enviarías a dirección?

15–20 min · sin instalar nada

- Reconstruye un pedido en los CSV.
- Revisa cuatro borradores reales del modelo.
- Corrige el mejor con los mínimos cambios.

Una cifra correcta no basta: mira a qué se refiere.

EJERCICIO 3

Construye una herramienta

~60 min · Python + Ollama

- Escribe la función cesta() y pasa las pruebas.
- Descríbela para el modelo y lanza seis preguntas.
- Haz un experimento: busca errores que no dan error.

Termina con un hallazgo que abrirá la sesión 2.

PARA LLEVAR

Tres ideas para recordar

1 **Un LLM redacta, no calcula.**
Las cifras las pone el código: pandas, una función fija o un SQL revisado.

2 **Local es control.**
Los datos no salen del equipo y no hay coste por uso.

3 **Verifica siempre: los datos que llegan y lo que escribe el modelo.**
El correo decía 20.000 €; el modelo dijo 8 SKU. Los dos sonaban seguros.

Próxima sesión: llegan los documentos internos. ¿Qué cambió el 30 de marzo?